UML Diagrams

What is UML?

- Standard language for specifying, visualizing, constructing, and documenting the artifacts of software systems, business modeling and other non-software systems.

- The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.

- The UML is a very important part of developing object oriented software and the software development process.

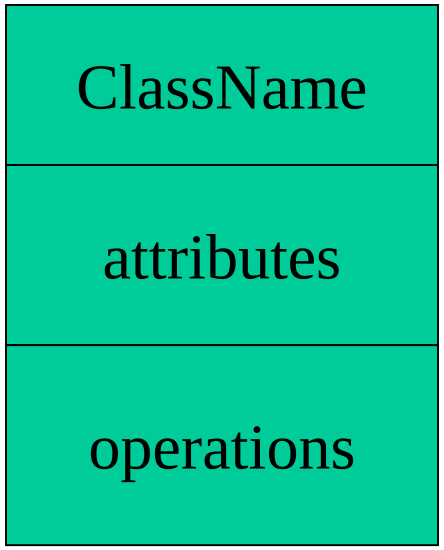
- Using the UML helps project teams communicate, explore potential designs, and validate the architectural design of the software.

UML Diagrams

- **Use-Case** *(relation of actors to system functions)*
- **Class** *(static class structure)*
- **Object** *(same as class - only using class instances – i.e. objects)*
- **State** *(states of objects in a particular class)*
- **Sequence** *(Object message passing structure)*
- **Collaboration** *(same as sequence but also shows context - i.e. objects and their relationships)*
- **Activity** *(sequential flow of activities i.e. action states)*
- **Component** *(code structure)*
- **Deployment** *(mapping of software to hardware)*

Class Diagrams

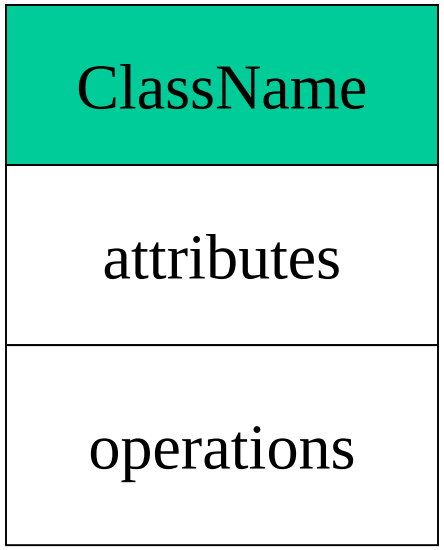
Classes



A *class* is a description of a set of objects that share the same attributes, operations, relationships, and semantics.

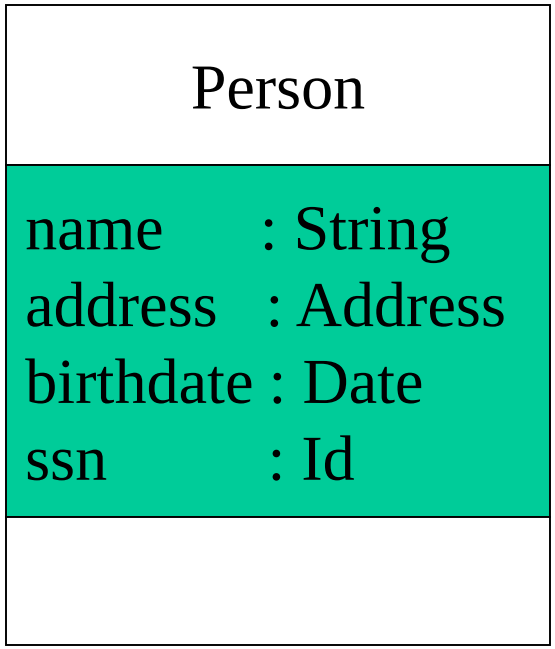
Graphically, a class is rendered as a rectangle, usually including its name, attributes, and operations in separate, designated compartments.

Class Names



The **name of the class** is the only required tag in the graphical representation of a class. It always appears in the top-most compartment.

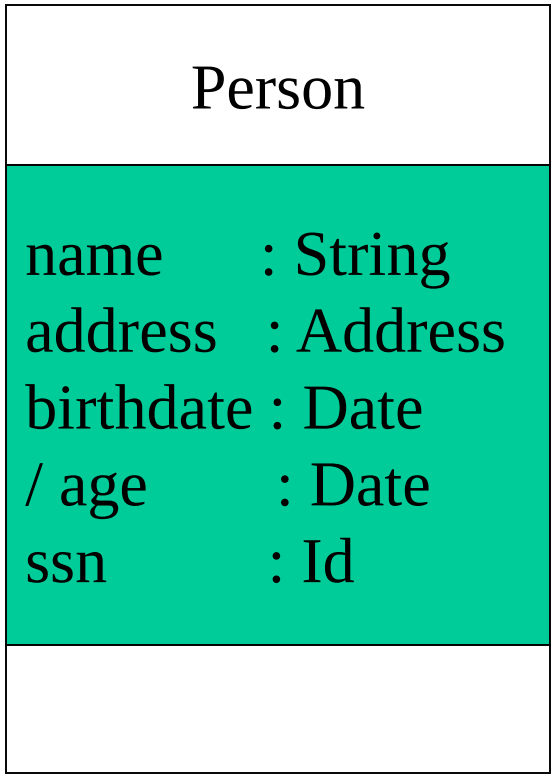
Class Attributes



An ***attribute*** is a named property of a class that **describes the object being modeled**.

In the class diagram, attributes appear in the second compartment just below the name-compartment.

Class Attributes (Cont'd)



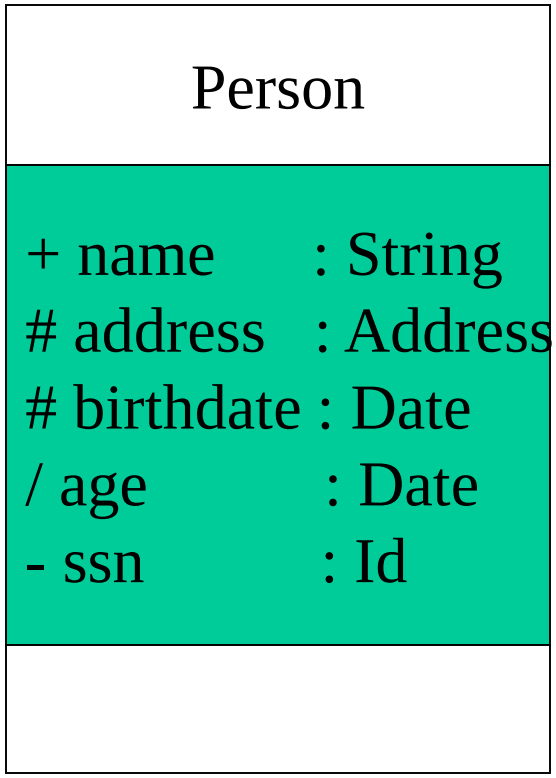
Attributes are usually listed in the form:

attributeName : Type

A *derived* attribute is one that can be computed from other attributes, but doesn't actually exist. For example, a Person's age can be computed from his birth date. A derived attribute is designated by a preceding '/' as in:

/ age : Date

Class Attributes (Cont'd)



Attributes can be:

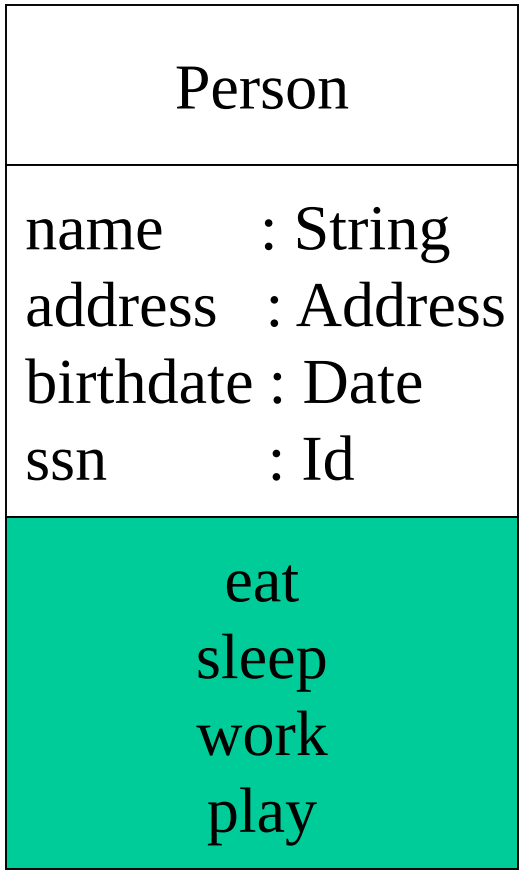
+ public

protected

- private

/ derived

Class Operations



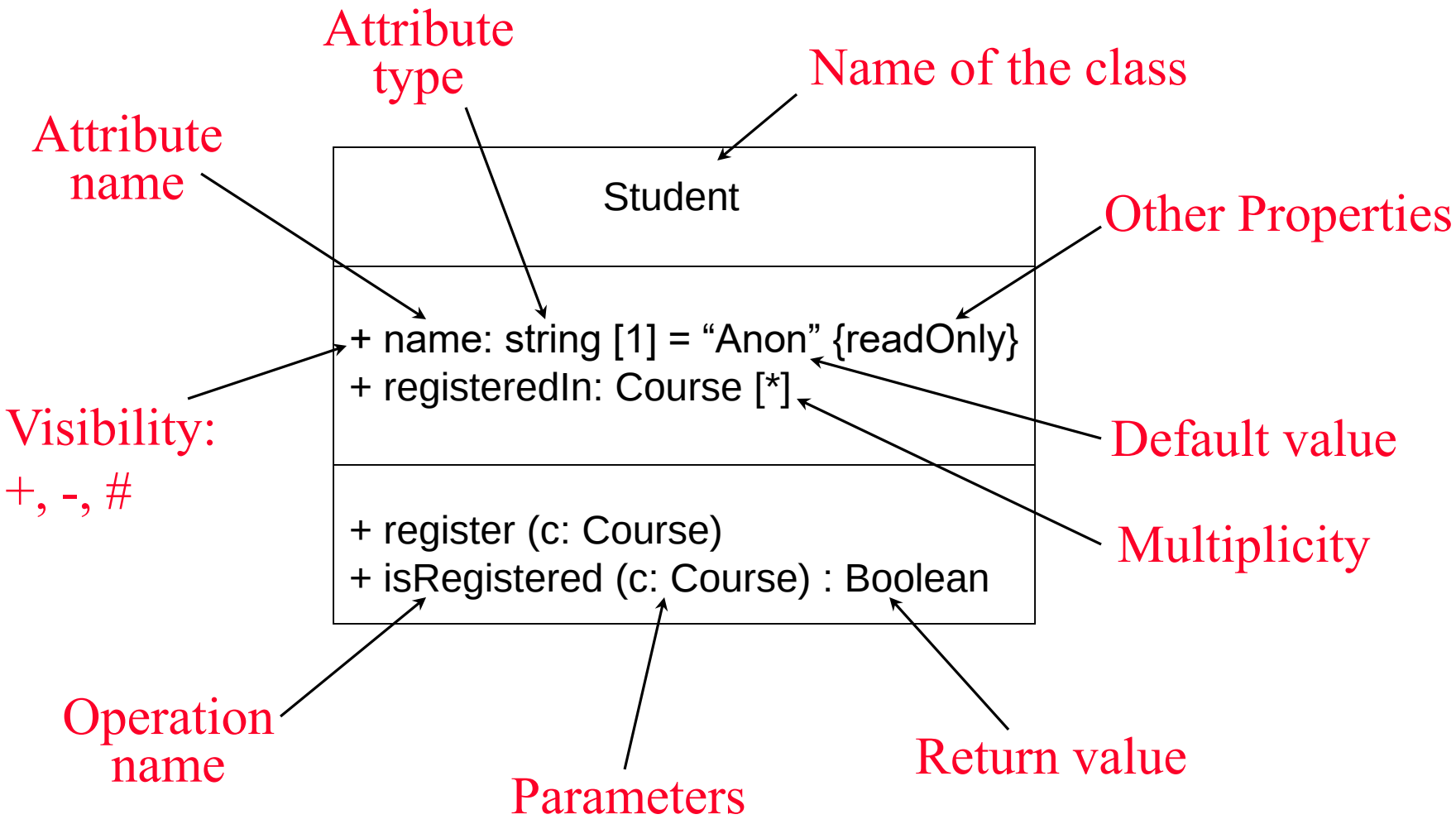
Operations describe the class behavior and appear in the third compartment.

Class Operations (Cont'd)

PhoneBook
newEntry (n : Name, a : Address, p : PhoneNumber, d : Description) getPhone (n : Name, a : Address) : PhoneNumber

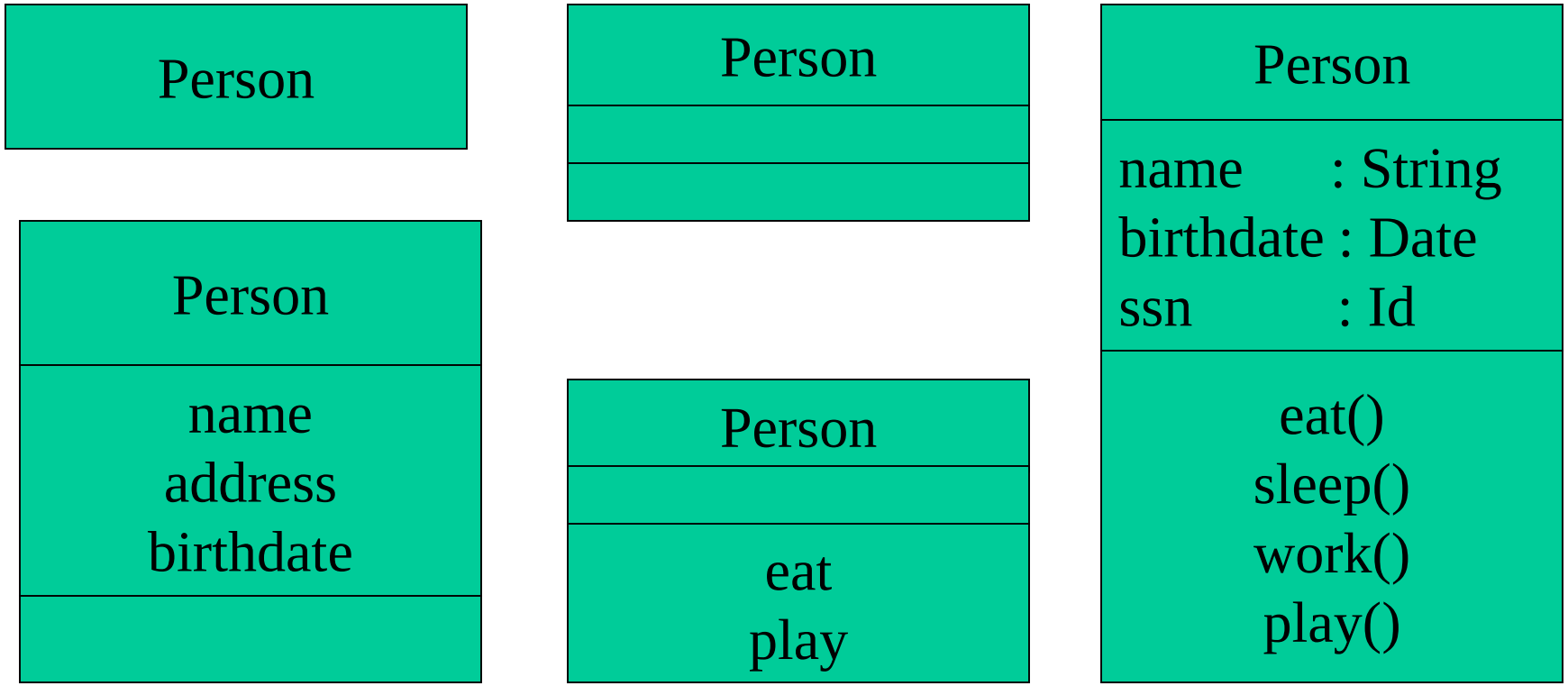
You can specify an operation by stating its signature: listing the name, type, and default value of all parameters, and, in the case of functions, a return type.

The full notation…



Depicting Classes

When drawing a class, you needn't show attributes and operation in every diagram.



UML Class-to-Java Example

```
Public class UNIXaccount
{
  public string username;
  public string groupname = "csai";
  public int filesystem_size;
  public date creation_date;
  private string password;
  static private integer no_of_accounts = 0
  public UNIXaccount()
  {
    //Other initialisation
    no_of_accounts++;
  }
  //Methods go here
};
```

UNIXaccount
+ username : string
+ groupname : string = “staff"
+ filesystem_size : integer
+ creation_date : date
- password : string
- no_of_accounts : integer = 0

Operations (Methods)

```
Public class Figure
{
  private int x = 0;
  private int y = 0;
  public void draw()
  {
    //Java code for drawing figure
  }
};
```

Figure
- x : integer = 0 - y : integer = 0
+ draw()

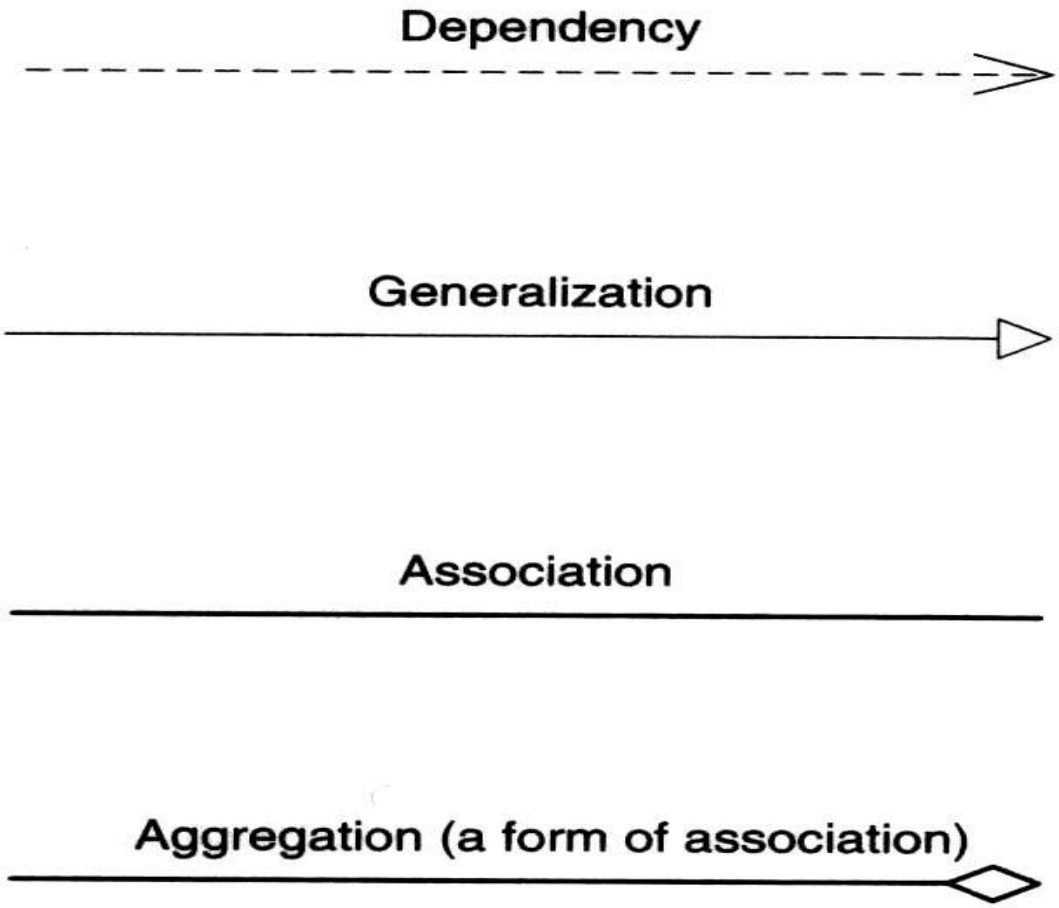
Relationships

In UML, object interconnections (logical or physical), are modeled as relationships.

There are three kinds of relationships in UML:

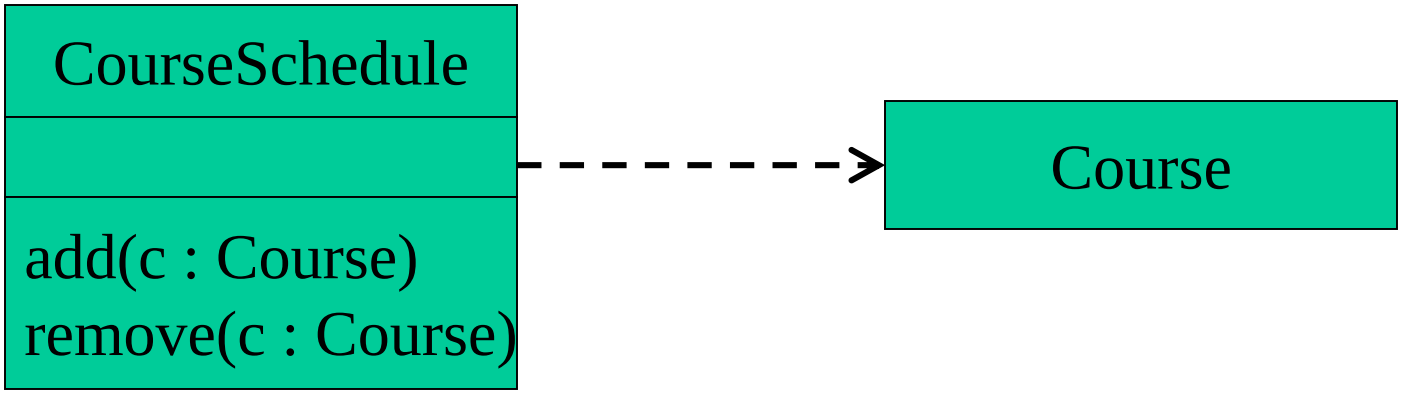
- dependencies
- generalizations
- associations

UML Relationships

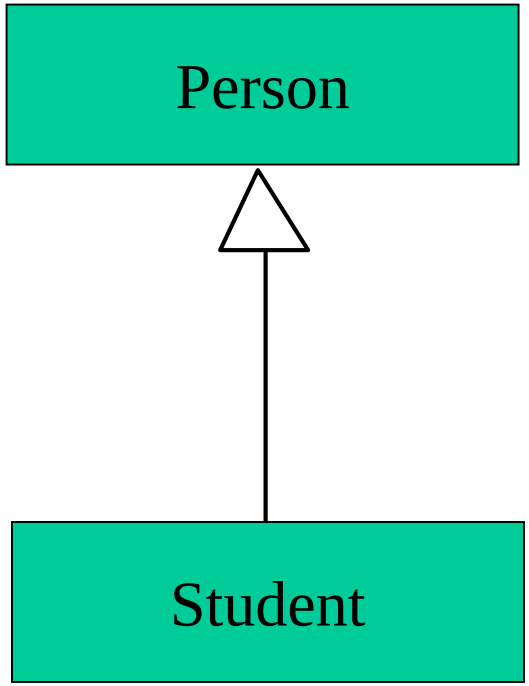


Dependency Relationships

A *dependency* indicates a semantic relationship between two or more elements. The dependency from *CourseSchedule* to *Course* exists because *Course* is used in both the **add** and **remove** operations of *CourseSchedule*.



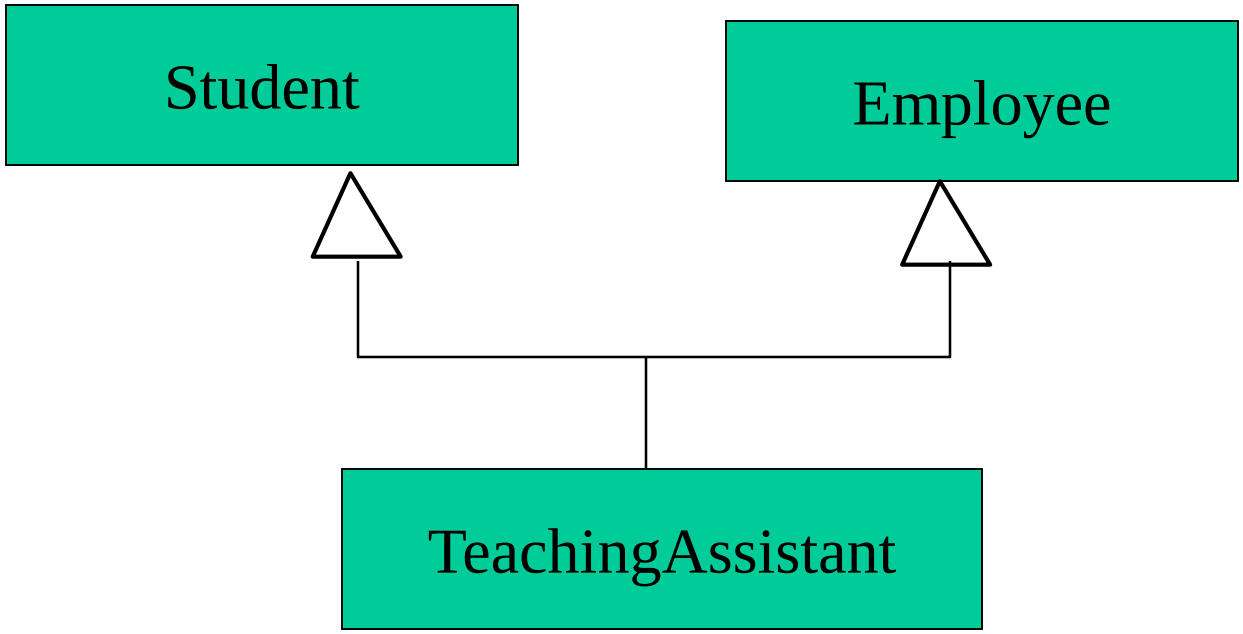
Generalization Relationships



A *generalization* connects a subclass to its superclass. It denotes an inheritance of attributes and behavior from the superclass to the subclass and indicates a specialization in the subclass of the more general superclass.

Generalization Relationships (Cont'd)

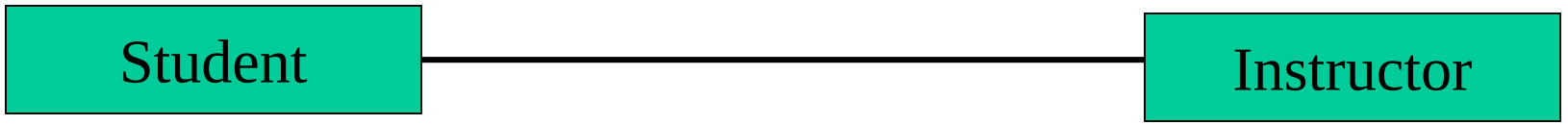
UML permits a class to inherit from multiple superclasses, although some programming languages (*e.g.*, Java) do not permit multiple inheritance.



Association Relationships

If two classes in a model need to communicate with each other, there must be link between them.

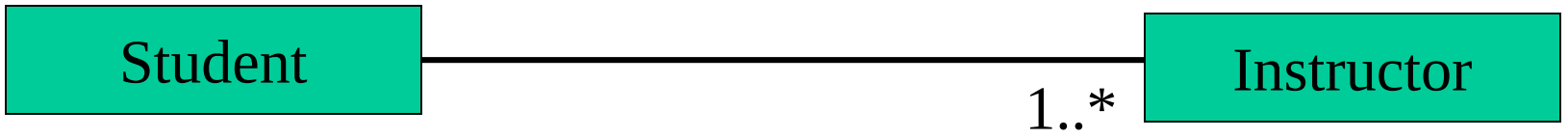
An *association* denotes that link.



Association Relationships (Cont'd)

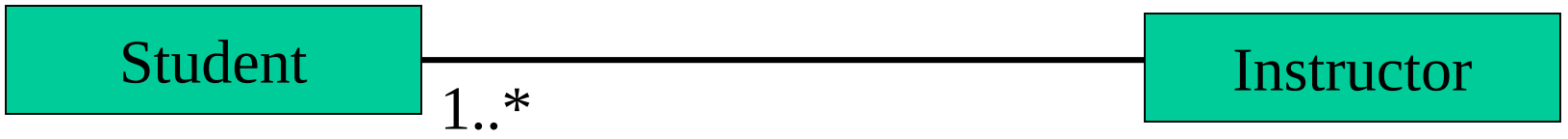
We can indicate the ***multiplicity*** of an association by adding *multiplicity adornments* to the line denoting the association.

The example indicates that a *Student* has one or more *Instructors*:



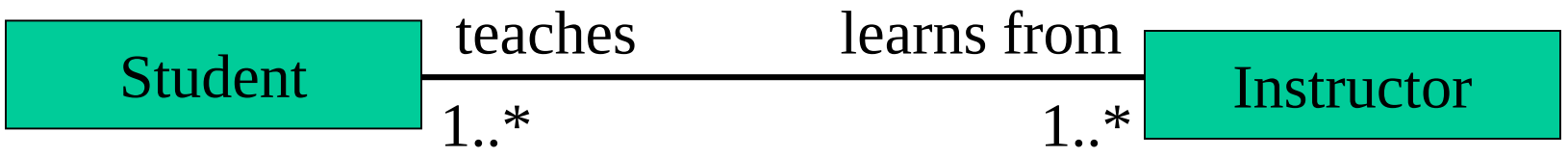
Association Relationships (Cont'd)

The example indicates that every *Instructor* has one or more *Students*:



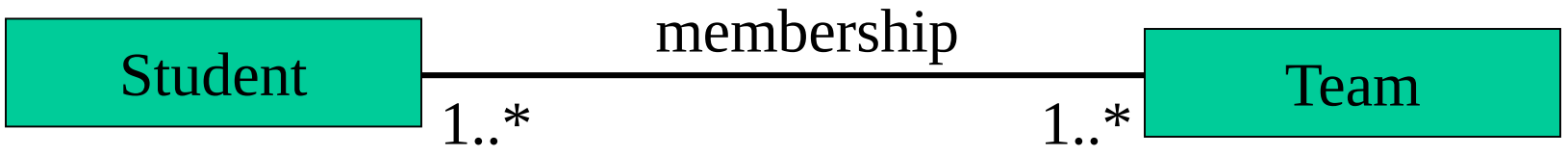
Association Relationships (Cont'd)

We can also indicate the behavior of an object in an association (*i.e.*, the *role* of an object) using *rolenames*.



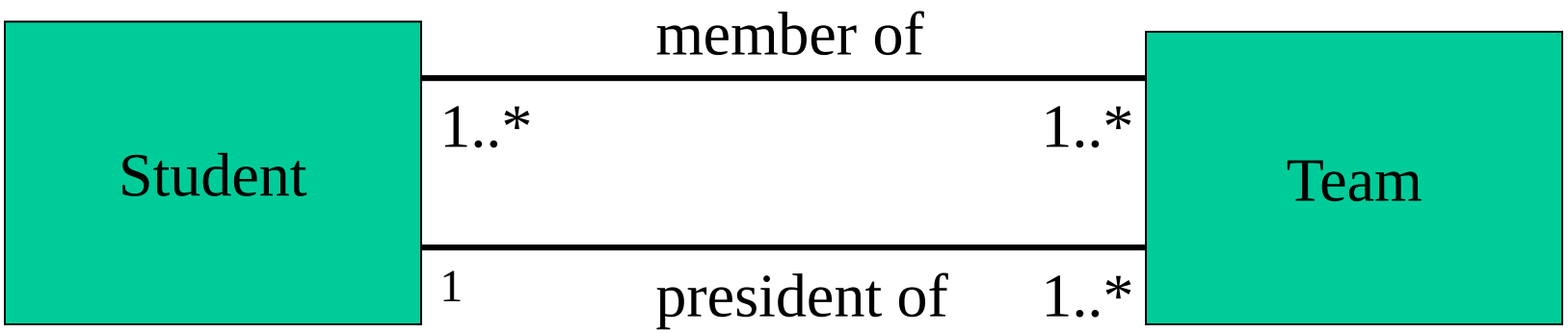
Association Relationships (Cont'd)

We can also name the association.



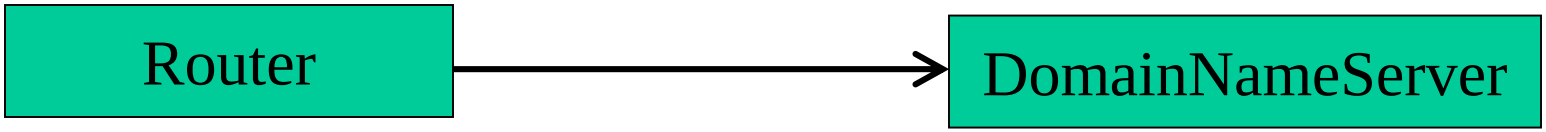
Association Relationships (Cont'd)

We can specify dual associations.



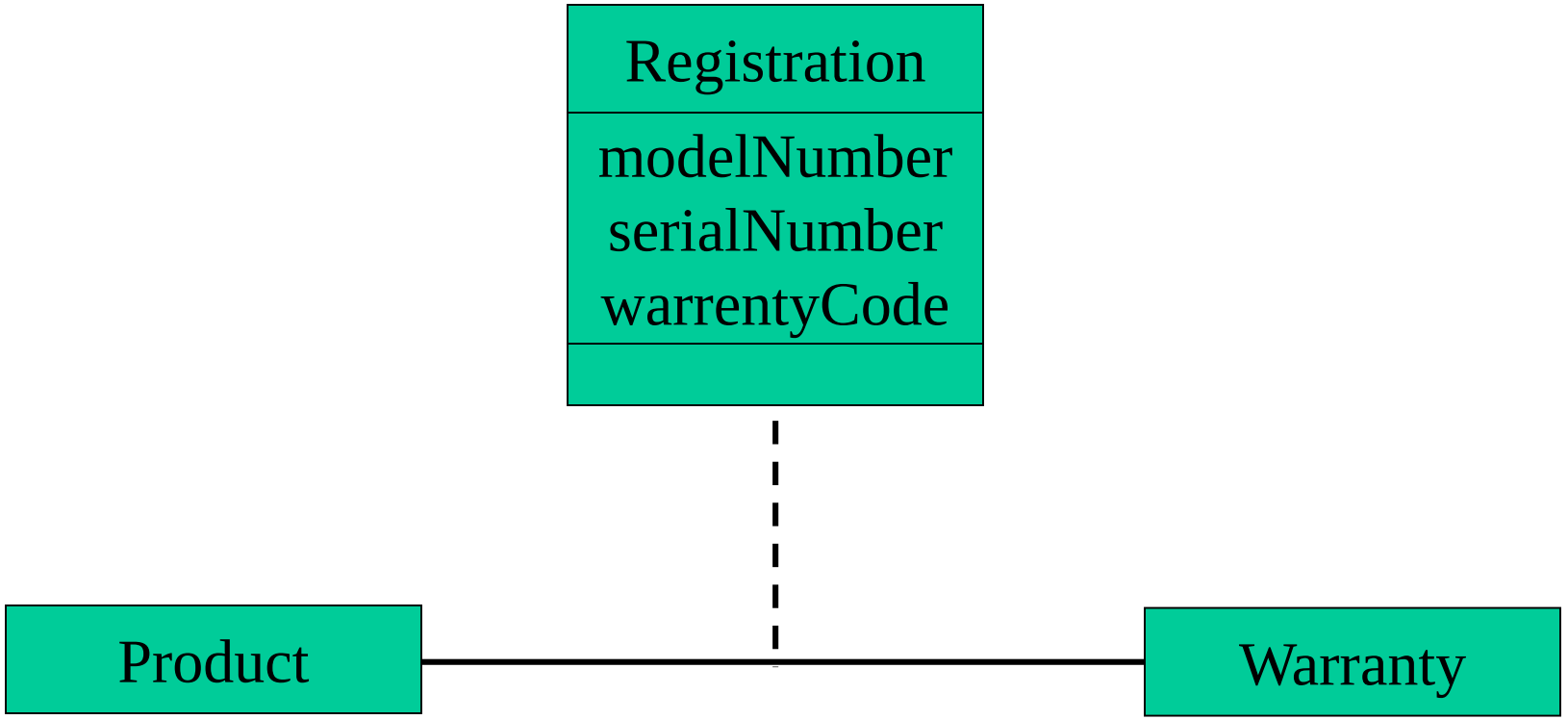
Association Relationships (Cont'd)

We can constrain the association relationship by defining the *navigability* of the association. Here, a *Router* object requests services from a *DNS* object by sending messages to (invoking the operations of) the server. **The direction of the association indicates that the server has no knowledge of the *Router*.**



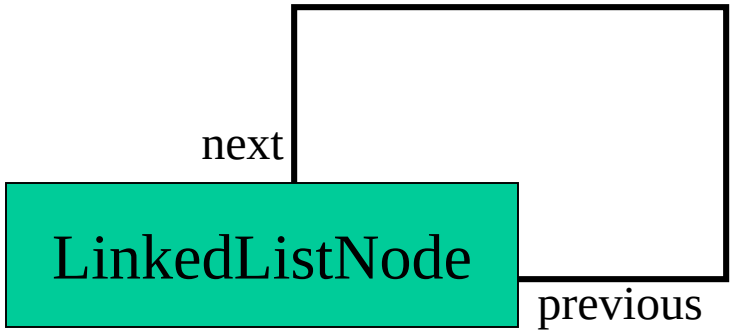
Association Relationships (Cont'd)

Associations can also be objects themselves, called *link classes* or an *association classes*.



Association Relationships (Cont'd)

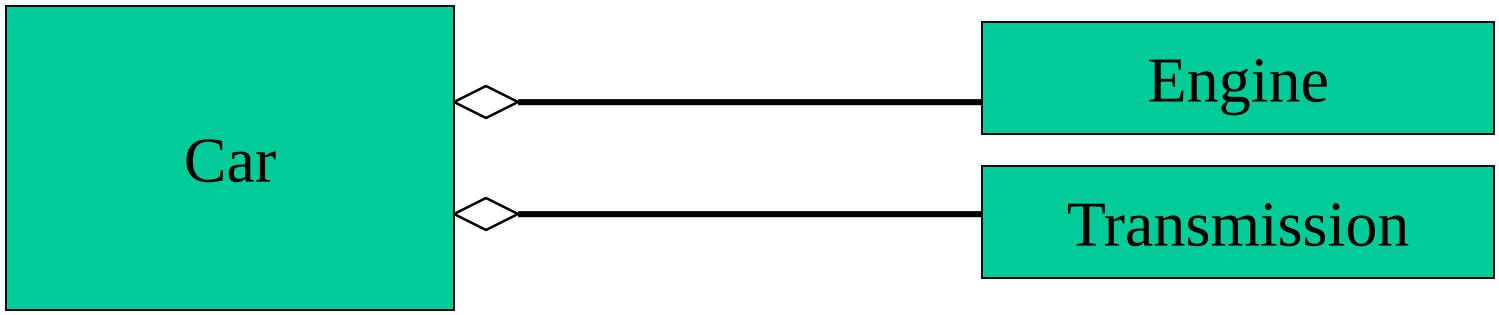
A class can have a *self association*.



Association Relationships (Cont'd)

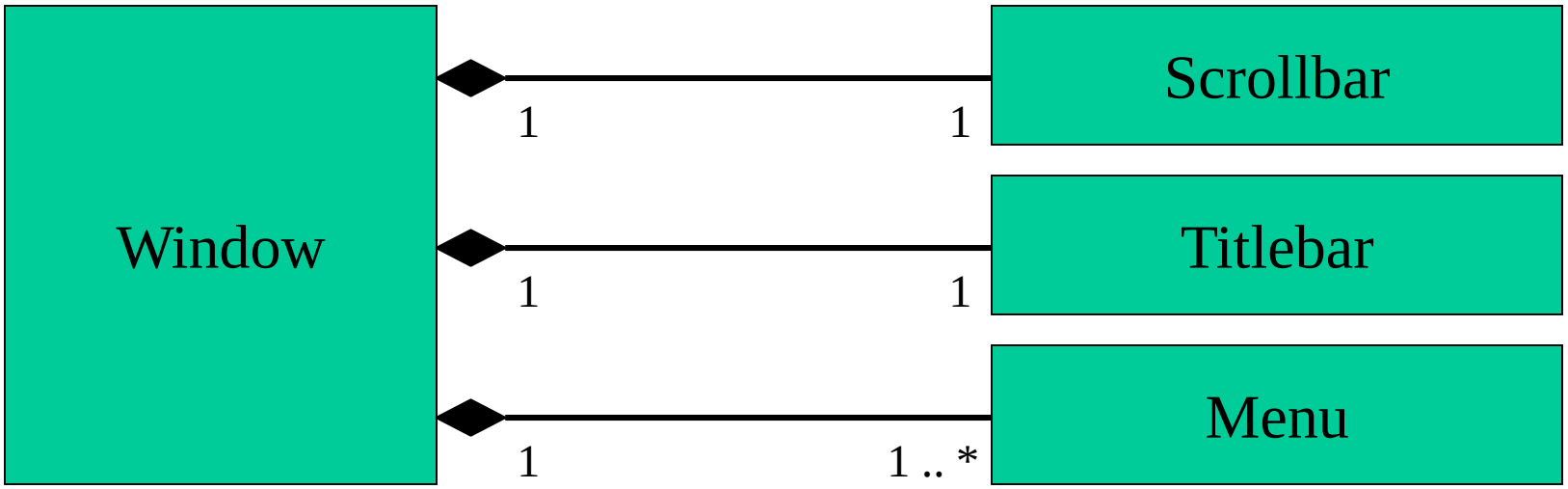
We can model objects that contain other objects by way of special associations called *aggregations* and *compositions*.

An *aggregation* specifies a whole-part relationship between an aggregate (a whole) and a constituent part, where the part can exist independently from the aggregate.
Aggregations are denoted by a hollow-diamond adornment on the association.



Association Relationships (Cont'd)

A *composition* indicates a strong ownership and coincident lifetime of parts by the whole (*i.e.*, they live and die as a whole). Compositions are denoted by a filled-diamond adornment on the association.

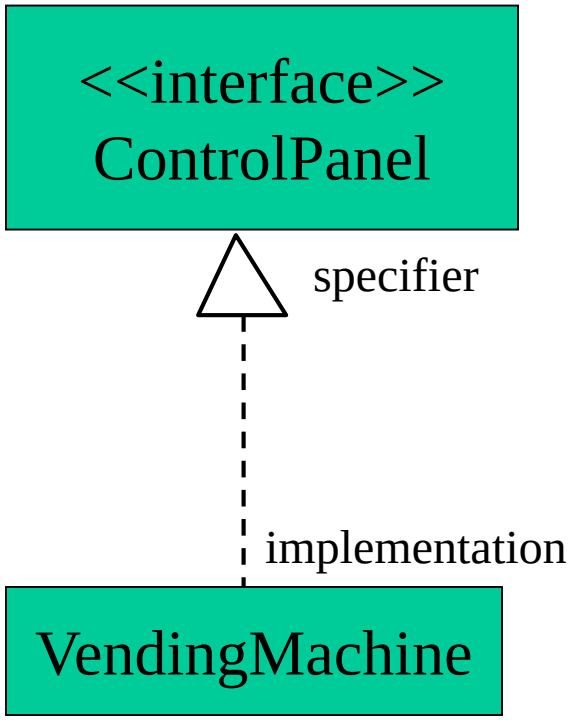


Interfaces

An *interface* is a named set of operations that specifies the behavior of objects without showing their inner structure.

It can be rendered in **the model by a one- or two-compartment rectangle**, with the *stereotype* <<interface>> above the interface name.

Interface Realization Relationship



A *realization* relationship connects a class with an interface that supplies its behavioral specification.

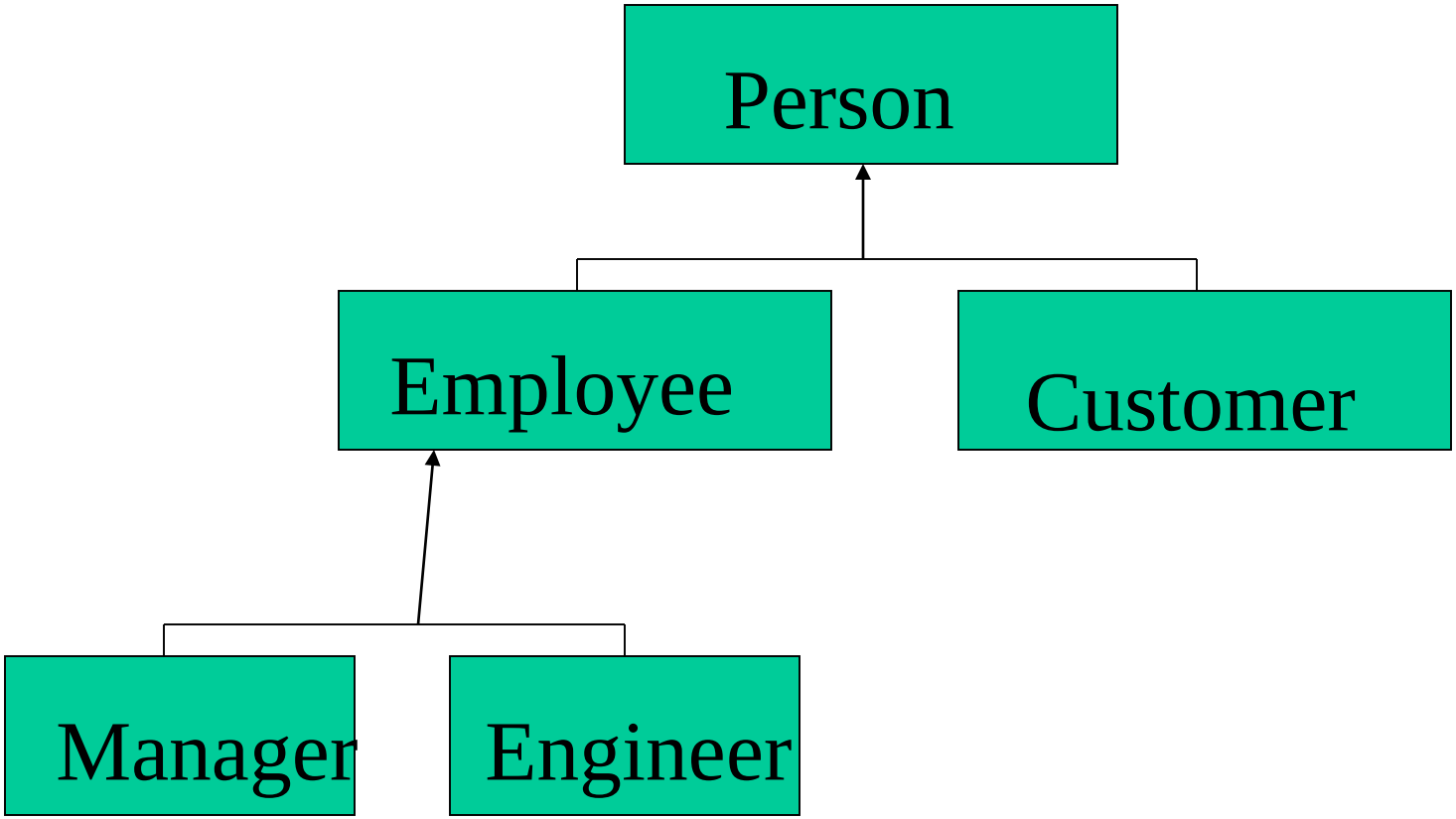
It is rendered by a dashed line with a hollow triangle towards the specifier.

Association (More Examples)

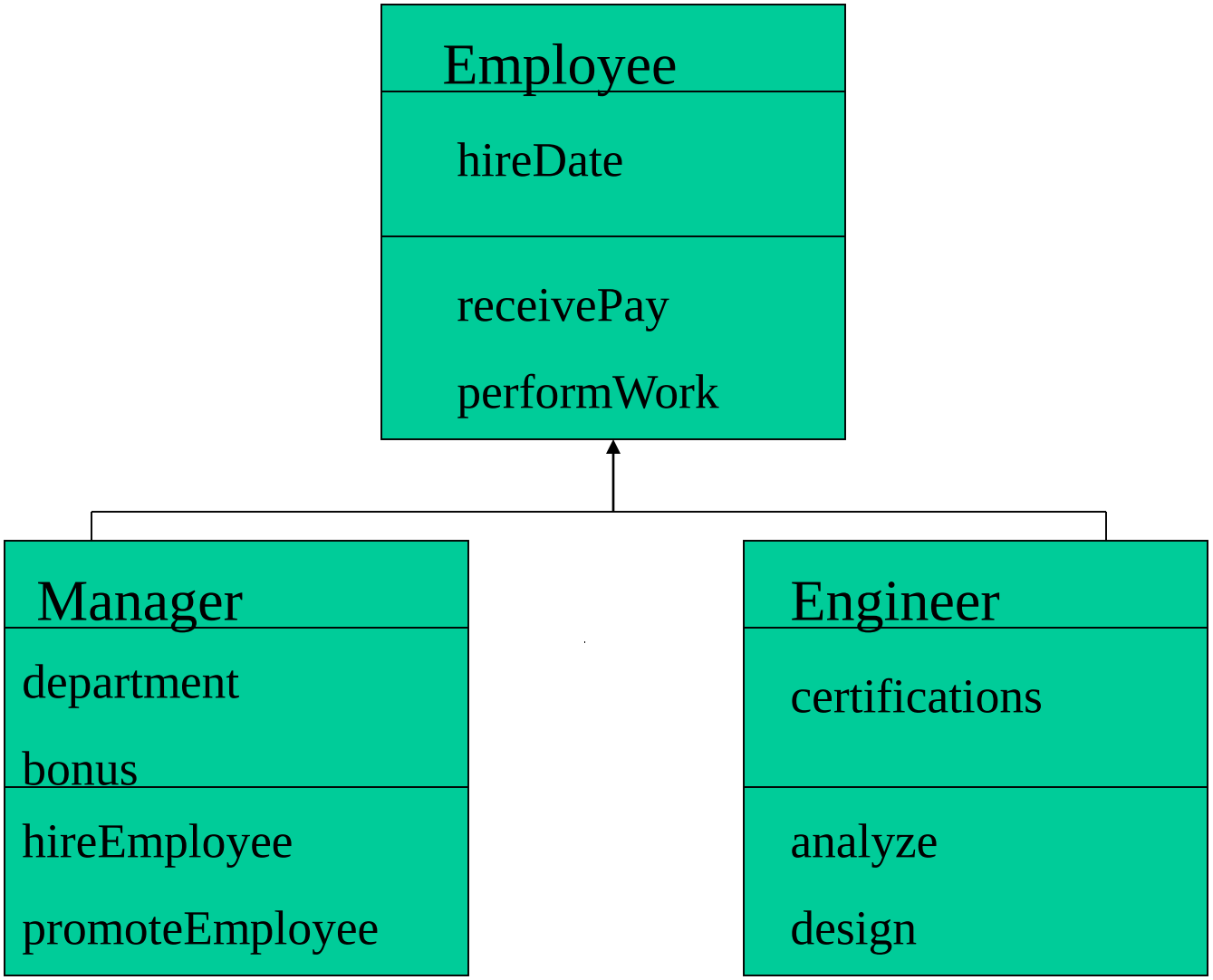
Association Relationship

- Name of relationship type shown by:
  - drawing line between classes
  - labeling with the name of the relationship
  - indicating with a small solid triangle beside the name of the relationship the direction of the association

Generalization Relationship



Generalization Relationship



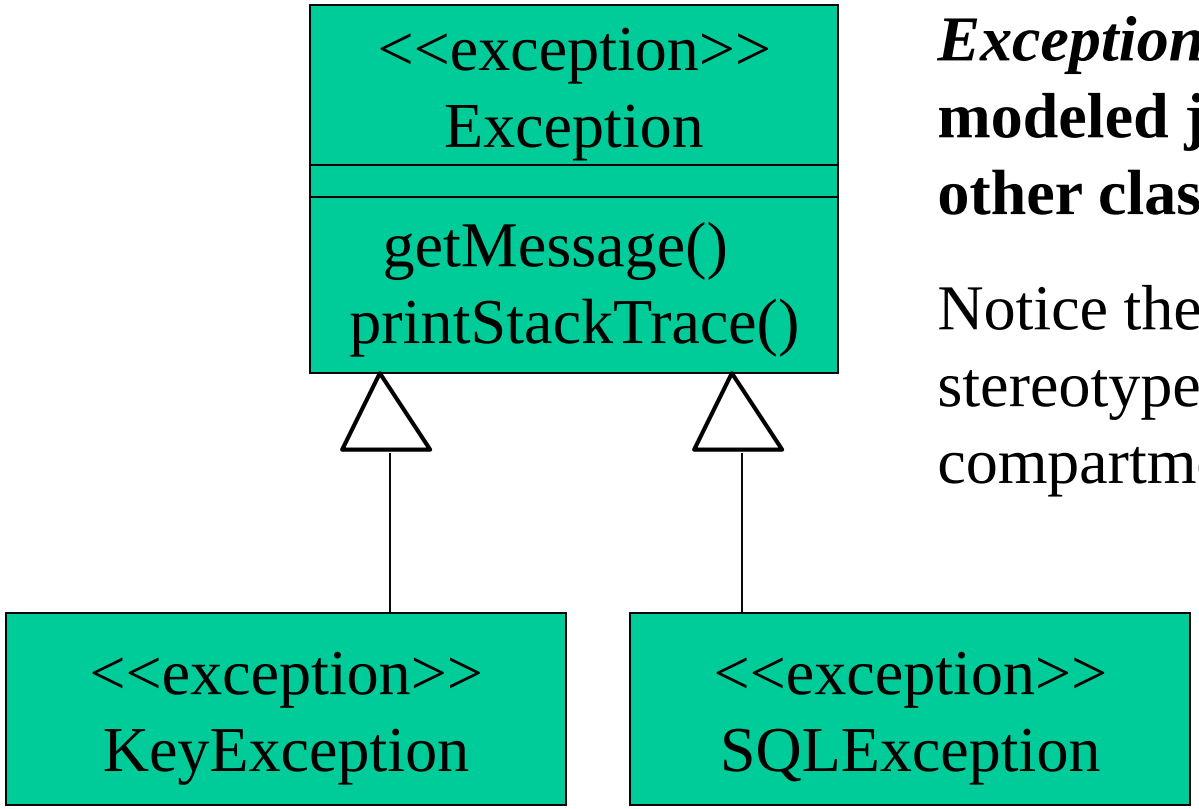
Aggregation Relationship

- Denoted by placing a diamond nearest the class representing the aggregation

Multiplicity

- Documents how many instances of a class can be associated with one instance of another class

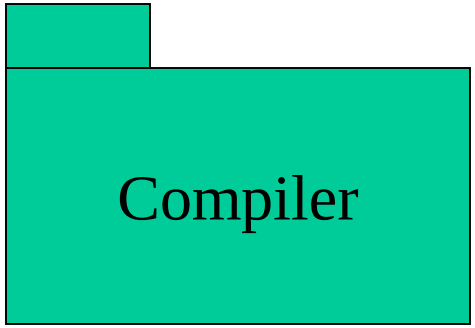
Exceptions



***Exceptions* can be modeled just like any other class.**

Notice the <<exception>> stereotype in the name compartment.

Packages



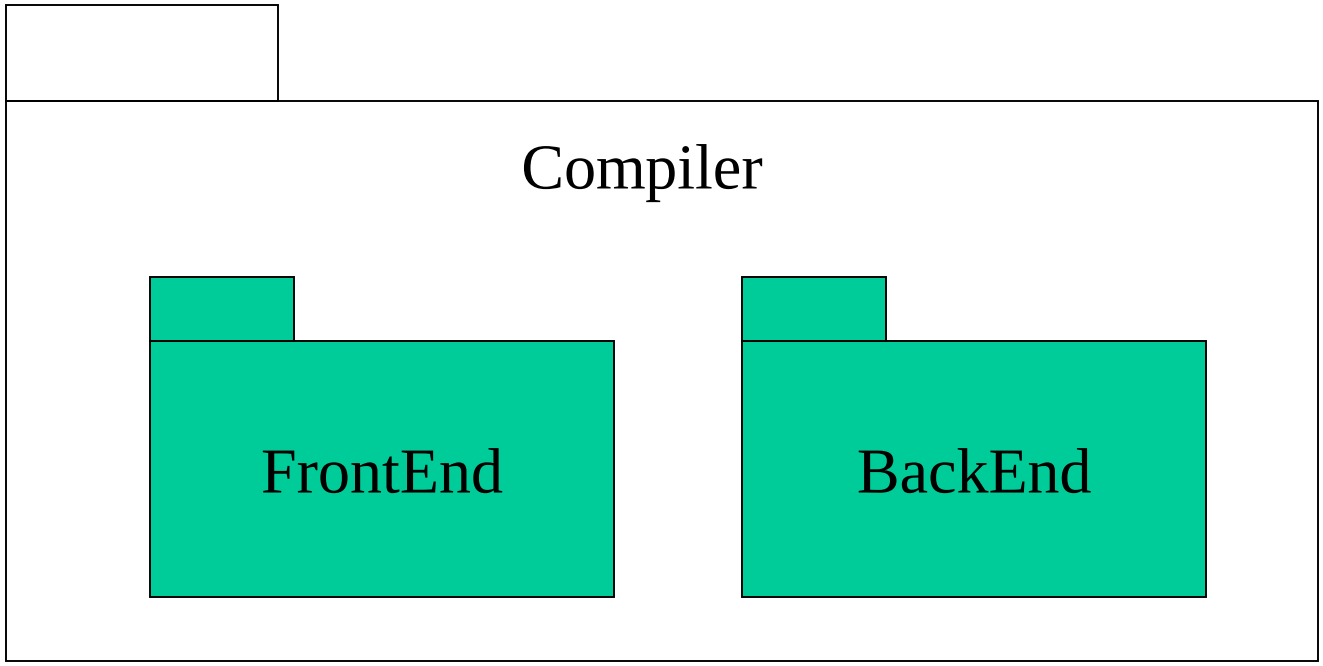
A *package* is a container-like element for organizing other elements into groups.

A package can contain classes and other packages and diagrams.

Packages can be used to provide controlled access between classes in different packages.

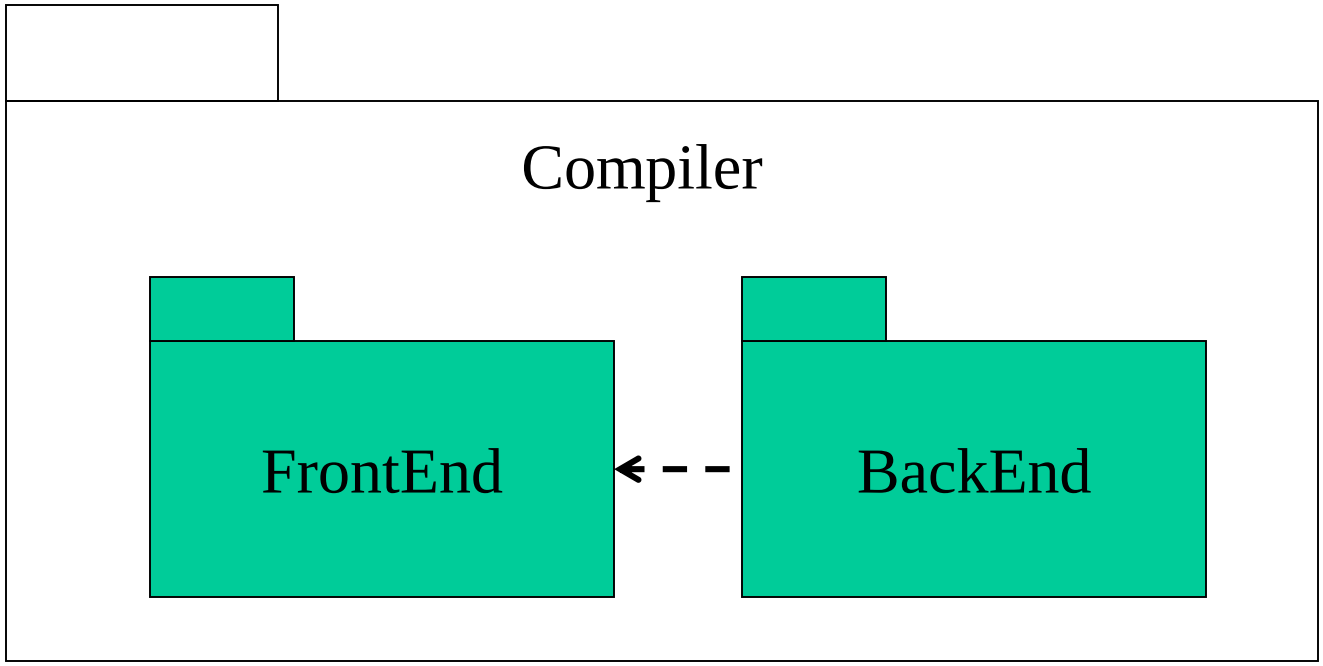
Packages (Cont'd)

Classes in the *FrontEnd* package and classes in the *BackEnd* package cannot access each other in this diagram.

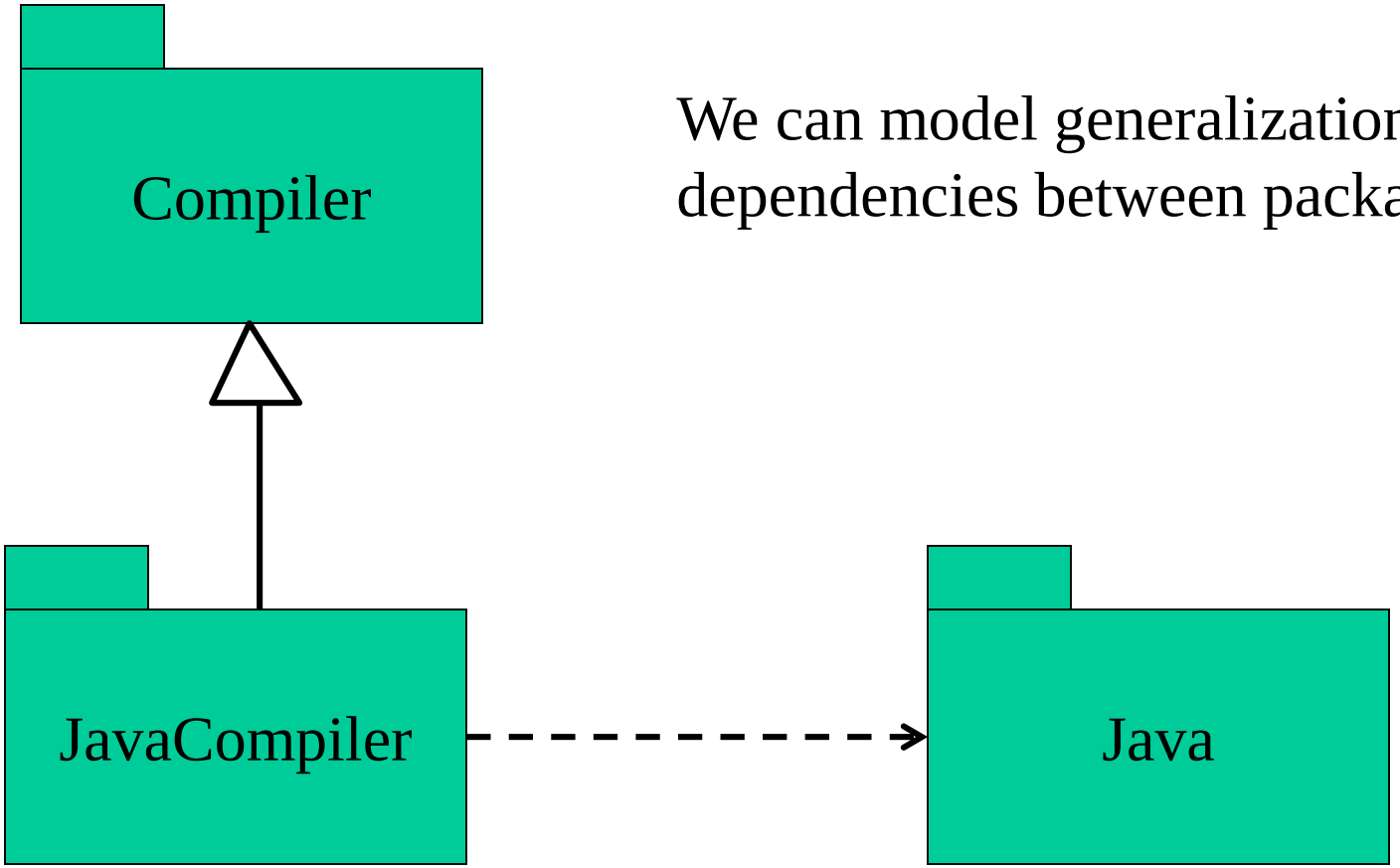


Packages (Cont'd)

Classes in the *BackEnd* package now have access to the classes in the *FrontEnd* package.



Packages (Cont'd)



We can model generalizations and dependencies between packages.